Level 5 Term 1 Project

Evaluation Rubric

Main Task 1: Hardware Simulation and Project Setup

Subtask 1.1: Circuit Design and Component Wiring in SimulIDE

Task Details:

1. Open SimulIDE and create a new circuit file.
2. Place all required components: ATmega32, DC motor, H-bridge, potentiometer, four push buttons (start, stop, emergency, and object sensor), status LEDs, and UART terminal.
3. Connect the motor direction pins and the PWM speed pin (PB3 / OC0) through the H-bridge to the DC motor.
4. Wire the potentiometer output to PA0 (ADC0).
5. Connect start and stop buttons to GPIO input pins with pull-up resistors.
6. Connect the emergency button to PD2 (INT0) and the object sensor button to PD3 (INT1).
7. Wire the UART TX pin (PD1) and RX pin (PD0) to the virtual serial terminal.
8. Label every component and wire clearly.
9. Take a clear screenshot of the full simulated circuit.

Subtask 1.2: Layered Project Setup in Microchip Studio

Task Details:

1. Create a new AVR XC8 C Application Project in Microchip Studio targeting ATmega32.
2. Name the project using the required naming convention.
3. Create three virtual folders inside the project: MCAL, HAL, and APP.
4. Add all MCAL driver source and header files to the MCAL folder: GPIO, EXTI, Timer, PWM, ADC, and UART.
5. Add all HAL driver source and header files to the HAL folder: DC Motor, Button, and Sensor HAL.
6. Configure the compiler include paths for MCAL, HAL, and APP so all headers resolve during build.
7. Verify all files appear in the Solution Explorer with no red errors.

Subtask 1.3: Build, HEX Generation, and Documentation

Task Details:

1. Build the complete project in Microchip Studio and resolve all compilation errors.
2. Locate the generated HEX file in the project Debug or Release output folder.
3. Load the HEX file into the ATmega32 in SimulIDE.
4. Power on the simulation and confirm the circuit initializes without crashes.
5. Prepare a pin-mapping table listing every function, its ATmega32 pin, and the driver used.
6. Write a short description for each hardware component explaining its role in the system.
7. Submit the circuit file, project folder, pin-mapping table, descriptions, and circuit screenshot.

Evaluation Criteria:

Task Instructions	Passing Criteria
Subtask 1.1: Circuit Design and Component Wiring in SimulIDE	
Create and name the SimulIDE circuit file	• Circuit file is created and saved with a clear name. • File naming follows the required convention.

Task Instructions	Passing Criteria
Place all required components	• ATmega32, DC motor, H-bridge, potentiometer, four buttons, status LEDs, and UART terminal are all present. • No required component is missing from the circuit.
Wire the motor circuit correctly	• Motor direction pins are connected through the H-bridge. • PWM speed pin (OC0 / PB3) is wired to the H-bridge enable input.
Wire all input components correctly	• Potentiometer output is connected to PA0 (ADC0). • Start and stop buttons are on GPIO input pins with pull-up resistors. • Emergency button is on PD2 (INT0) and object sensor button is on PD3 (INT1).
Wire the UART terminal	• TX (PD1) and RX (PD0) are connected to the virtual serial terminal. • Terminal baud rate matches the UART driver configuration.
Labels and documentation	• Every component has a clear label. • Screenshot shows the full circuit with all labels readable.
Subtask 1.2: Layered Project Setup in Microchip Studio	
Create the Microchip Studio project	• Project targets the ATmega32 device correctly. • Project is named using the required naming convention.
Three-layer folder structure	• MCAL, HAL, and APP folders are created inside the project. • Every driver file is placed in its correct layer. • No driver is placed in the wrong folder.
MCAL drivers present	• GPIO, EXTI, Timer, PWM, ADC, and UART .c and .h files are in the MCAL folder. • No MCAL driver is missing.
HAL drivers present	• DC Motor, Button, and Sensor HAL .c and .h files are in the HAL folder. • No HAL driver is missing.
Include paths configured	• Include paths are set for MCAL, HAL, and APP in the project properties. • All headers resolve without errors after configuration.
Subtask 1.3: Build, HEX Generation, and Documentation	
Project builds successfully	• Project compiles with no errors. • All source files are included and linked correctly.
HEX file generated and loaded	• HEX file is present in the project output folder. • HEX loads into the ATmega32 in SimulIDE without errors. • Simulation powers on and the circuit initializes.
Pin-mapping table	• Table covers every hardware component. • Each row includes: function name, ATmega32 pin, and driver used. • No pin is missing or incorrectly assigned.

Task Instructions	Passing Criteria
Component descriptions	• A short description is provided for every component. • Each description explains the component's role in the conveyor system.
All deliverables submitted	• SimulIDE circuit file, Microchip Studio project, pin-mapping table, descriptions, and screenshot are all submitted.

Main Task 2: System Flags and Motor Control

Subtask 2.1: Define System Flags

Task Details:

1. At the top of main.c, declare global volatile flag variables of type u8 for every system event.
2. Define the following flags: Flag_StartPressed, Flag_StopPressed, Flag_EmergencyStop, Flag_ConveyorRunning, Flag_ObjectDetected, and Flag_SpeedChanged.
3. Initialize all flags to 0 at the start of the program.
4. Write a short comment next to each flag explaining when it is set and when it is cleared.
5. Inside the INT0 ISR (emergency button), set Flag_EmergencyStop to 1 immediately. Do not clear it inside the ISR.
6. Inside the INT1 ISR or GPIO polling logic (object sensor button), set Flag_ObjectDetected to 1.
7. In the main loop, check Flag_EmergencyStop first before checking any other flag.

Subtask 2.2: Motor Start, Stop, and Direction Control

Task Details:

1. Initialize the DC motor driver at the start of the program using MOTOR_voidInit.
2. Set the motor direction to clockwise using MOTOR_voidSetDirectionCW before starting the conveyor.
3. In the main loop, check if Flag_StartPressed is 1 and Flag_ConveyorRunning is 0 to start the motor.
4. When starting: call MOTOR_voidSetSpeed with an initial duty cycle, set Flag_ConveyorRunning to 1, clear Flag_StartPressed, and send "Conveyor Running" over UART.
5. In the main loop, check if Flag_StopPressed is 1 and Flag_ConveyorRunning is 1 to stop the motor.
6. When stopping: call MOTOR_voidSetSpeed(0) to cut the PWM, set Flag_ConveyorRunning to 0, clear Flag_StopPressed, and send "Conveyor Stopped" over UART.
7. Poll the start and stop GPIO buttons inside the main loop and set their flags when a press is detected.

Subtask 2.3: Speed Control using ADC and PWM

Task Details:

1. Initialize the ADC driver using ADC_voidInit with AVCC reference and prescaler 128.
2. Initialize the PWM driver using PWM_voidInit on PWM_CHANNEL_0 with fast mode and non-inverting polarity.
3. Inside the main loop, while Flag_ConveyorRunning is 1, read the potentiometer using ADC_u16Read on ADC_CHANNEL_0.
4. Divide the 10-bit ADC result (0 to 1023) into three speed levels: Low (0-340), Medium (341-681), High (682-1023).
5. Map each level to a PWM duty cycle: Low = 85, Medium = 170, High = 255.
6. If the speed level has changed since the last reading, call MOTOR_voidSetSpeed with the new duty cycle and set Flag_SpeedChanged to 1.
7. When Flag_SpeedChanged is 1, send the current speed level over UART (e.g. "Speed Level: High") and clear the flag.

Evaluation Criteria:

Task Instructions	Passing Criteria
Subtask 2.1: Define System Flags	

Task Instructions	Passing Criteria
Flag declarations	• All six flags are declared as volatile u8 at global scope. • All flags are initialized to 0 before the main loop. • Each flag has a comment explaining its purpose.
ISR flag usage	• Flag_EmergencyStop is set to 1 inside the INT0 ISR. • Flag_ObjectDetected is set to 1 inside the INT1 ISR or polling function. • No flag is cleared inside an ISR.
Main loop flag check order	• Emergency flag is checked first, before all other flags. • While Flag_EmergencyStop is 1, all other system actions are blocked. • Flag_EmergencyStop is never cleared by software. Only a hardware reset can clear it.
Flag clarity and correctness	• Each flag name clearly describes the event it represents. • No flag is used for two different purposes. • Flags are only set and cleared at the correct points in the code.
Subtask 2.2: Motor Start, Stop, and Direction Control	
Motor initialization	• MOTOR_voidInit is called once at startup. • Motor direction is set to clockwise before any movement begins.
Start logic using flags	• Motor starts only when Flag_StartPressed is 1 AND Flag_ConveyorRunning is 0. • Flag_ConveyorRunning is set to 1 after the motor starts. • Flag_StartPressed is cleared after handling. • "Conveyor Running" is sent over UART.
Stop logic using flags	• Motor stops only when Flag_StopPressed is 1 AND Flag_ConveyorRunning is 1. • Flag_ConveyorRunning is set to 0 after the motor stops. • Flag_StopPressed is cleared after handling. • "Conveyor Stopped" is sent over UART.
Button polling	• Start and stop buttons are polled correctly inside the main loop. • Each button press sets its corresponding flag exactly once per press. • Basic debouncing prevents repeated triggering from one press.
Subtask 2.3: Speed Control using ADC and PWM	
ADC and PWM initialization	• ADC is initialized with AVCC reference and prescaler 128. • PWM channel 0 is initialized in fast mode with non-inverting polarity.
ADC reading in the main loop	• ADC is read only when Flag_ConveyorRunning is 1. • Potentiometer value is read on ADC_CHANNEL_0 correctly.
Speed level mapping	• ADC range (0-1023) is divided into exactly three levels. • Each level maps to a correct PWM duty cycle value. • MOTOR_voidSetSpeed is called with the new duty cycle when the level changes.

Task Instructions	Passing Criteria
Flag_SpeedChanged usage	• Flag_SpeedChanged is set to 1 only when the speed level actually changes. • UART transmits the new speed level when the flag is set. • Flag_SpeedChanged is cleared after the UART message is sent. • Speed level is not re-printed when the ADC reading stays in the same level.

Main Task 3: Object Detection, Logging, and Emergency Handling

Subtask 3.1: Object Detection and Counting

Task Details:

1. Configure the object sensor button on PD3 (INT1) as a falling-edge EXTI interrupt using EXTI_voidInit and EXTI_voidSetCallBack.
2. In the INT1 ISR callback, set Flag_ObjectDetected to 1 only. Do not increment the counter inside the ISR.
3. Apply basic software debouncing in the ISR callback using a time-gap check or a simple flag guard.
4. In the main loop, when Flag_ObjectDetected is 1: increment the object counter, transmit "Object Detected - Count: N" over UART, and clear the flag.
5. The object counter must be a global u8 or u16 variable initialized to 0 at startup.
6. Object counting must only work when Flag_ConveyorRunning is 1. Ignore detections while the belt is stopped.
7. Test by pressing the object button multiple times and verifying the count increments correctly in the terminal.

Subtask 3.2: UART System Logging

Task Details:

1. Initialize the UART driver using UART_voidInit at 9600 baud, no parity, one stop bit.
2. At startup, before the main loop begins, transmit "System Ready" to confirm the system is alive.
3. Log every state change: send "Conveyor Running" when the motor starts, "Conveyor Stopped" when it stops.
4. Log every object detection: send "Object Detected - Count: N" where N is the current count.
5. Log every speed change: send "Speed Level: Low", "Speed Level: Medium", or "Speed Level: High".
6. Log the emergency event: send "Emergency Stop Activated" immediately when Flag_EmergencyStop is detected.
7. Every message must end with a carriage return and newline (\r\n) so each event appears on its own line in the terminal.

Subtask 3.3: Emergency Stop Handling

Task Details:

1. Configure the emergency button on PD2 (INT0) as a falling-edge EXTI interrupt using EXTI_voidInit and EXTI_voidSetCallBack.
2. Inside the INT0 ISR callback: set Flag_EmergencyStop to 1, stop the motor immediately by calling MOTOR_voidSetSpeed(0), and set Flag_ConveyorRunning to 0.
3. Turn on the red status LED (GPIO_HIGH) when the emergency flag is first detected in the main loop.
4. Transmit "Emergency Stop Activated\r\n" over UART immediately when Flag_EmergencyStop is detected in the main loop.
5. Once Flag_EmergencyStop is 1, block all start, stop, and speed actions in the main loop. Only the emergency condition is active.
6. Never clear Flag_EmergencyStop in software. The system remains locked until a hardware reset is performed.
7. Test the full sequence: run the conveyor, press the emergency button, verify the motor stops immediately, the red LED turns on, and the correct message appears in the terminal.

Evaluation Criteria:

Task Instructions	Passing Criteria
Subtask 3.1: Object Detection and Counting	
INT1 configuration	• Object sensor button is configured on INT1 (PD3) with falling-edge trigger. • ISR callback is registered using EXTI_voidSetCallBack. • Global interrupts are enabled with sei().
Flag usage in the ISR	• Only Flag_ObjectDetected is set inside the ISR. • Counter is never incremented inside the ISR. • Flag is not cleared inside the ISR.
Debouncing	• A debounce mechanism prevents a single press from triggering multiple counts. • One physical press results in exactly one count increment.
Counter logic in the main loop	• Counter increments only when Flag_ObjectDetected is 1 AND Flag_ConveyorRunning is 1. • Counter value stays accurate across multiple presses. • Flag_ObjectDetected is cleared after the counter is updated. • "Object Detected - Count: N" is printed with the correct N value.
Subtask 3.2: UART System Logging	
UART initialization	• UART is initialized at 9600 baud, no parity, one stop bit. • "System Ready\r\n" is the first message printed before the main loop begins.
State change logging	• "Conveyor Running\r\n" is transmitted when the motor starts. • "Conveyor Stopped\r\n" is transmitted when the motor stops. • Messages appear immediately when the event occurs.
Object and speed logging	• "Object Detected - Count: N\r\n" is transmitted after each detection with the correct count. • "Speed Level: X\r\n" is transmitted each time the speed level changes. • No duplicate messages are printed for the same event.
Message format	• Every message ends with \r\n. • Messages are readable and clearly identify the event. • All log messages appear in the correct order in the terminal.
Subtask 3.3: Emergency Stop Handling	
INT0 configuration	• Emergency button is on PD2 (INT0) with falling-edge trigger. • ISR callback is registered using EXTI_voidSetCallBack. • Global interrupts are enabled with sei().
Motor stop inside the ISR	• MOTOR_voidSetSpeed(0) is called inside the ISR callback without any delay. • Flag_EmergencyStop is set to 1 inside the ISR. • Flag_ConveyorRunning is set to 0 inside the ISR.

Task Instructions	Passing Criteria
Main loop emergency response	• Emergency flag is checked first before all other flags in the main loop. • Red LED turns ON when Flag_EmergencyStop is detected. • "Emergency Stop Activated\r\n" is transmitted over UART. • All other system actions (start, stop, speed) are blocked while the flag is active.
Latching behavior	• Flag_EmergencyStop is never cleared in software. • The start button has no effect while the emergency flag is active. • Only a hardware reset restores normal system operation.
Full flow testing	• Motor stops immediately when the emergency button is pressed. • Red LED turns on correctly. • UART terminal shows "Emergency Stop Activated" at the correct point in the log. • System remains locked and does not respond to the start button after emergency.

Submission Checklist

[ ] SimulIDE circuit file with all components correctly wired

[ ] Microchip Studio project folder (MCAL, HAL, APP structure)

[ ] Pin-mapping table

[ ] Short description of each component's role

[ ] Screenshot of the full simulated circuit

[ ] Generated HEX file

[ ] Full application code (main.c with flags and all driver calls)

[ ] Screenshot or short recording of the running simulation

[ ] UART terminal log screenshot showing: System Ready, Conveyor Running, Object Detected, Speed Level, Conveyor Stopped, Emergency Stop Activated